1. INTRODUCTION TO GIT

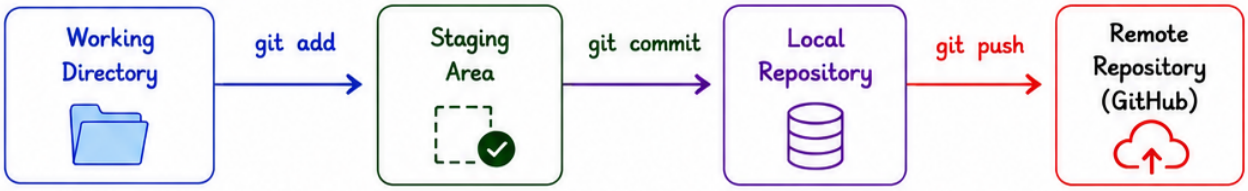
1.1 What is Git?

Git is a Distributed Version Control System (DVCS) that tracks changes in source code and helps developers collaborate efficiently without overwriting each other's work.

Why do we use Git?

- Keeps track of every change
- Restores previous versions
- Enables team collaboration
- Supports multiple branches
- Maintains project history
- Works Offline
- Secure and Reliable
- Widely used in Industry

1.2 Git Workflow



1.3 Key Components

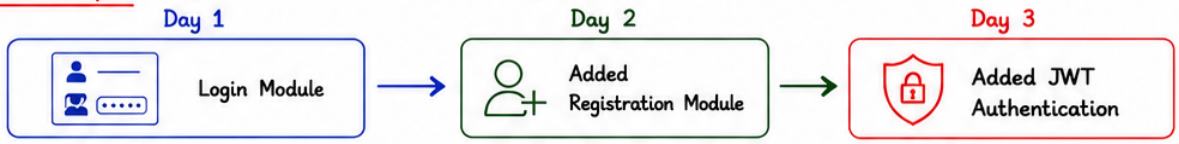
Component	Purpose
Working Directory	Where you create and edit files.
Staging Area	Holds selected changes before committing.
Local Repository	Stores the commit history on your local machine.
Remote Repository	Stores the project online (e.g., GitHub, GitLab, Bitbucket).

1.4 Advantages of Git

- Fast and Efficient
- Free and Open Source
- Distributed System
- Secure
- Lightweight
- Supports Branching
- Easy Collaboration
- Works Offline
- Handles Large Projects
- Cross Platform

1.5 Example

Suppose you are building a Banking Application.



Git stores every version, allowing you to return to any previous state whenever needed.

1.6 Interview Question

Q: Why is Git called a Distributed Version Control System?

Answer:

Because every developer has a complete copy of the repository, including the entire commit history. Developers can commit, branch, and merge locally without requiring constant access to a central server.

1.7 Remember

Git = Version Control System
GitHub = Cloud platform used to host Git repositories.

Quick Revision

- ✓ Track Changes
- ✓ Save Project History
- ✓ Collaborate with Teams
- ✓ Restore Previous Versions
- ✓ Push Code to GitHub

2. GIT ARCHITECTURE

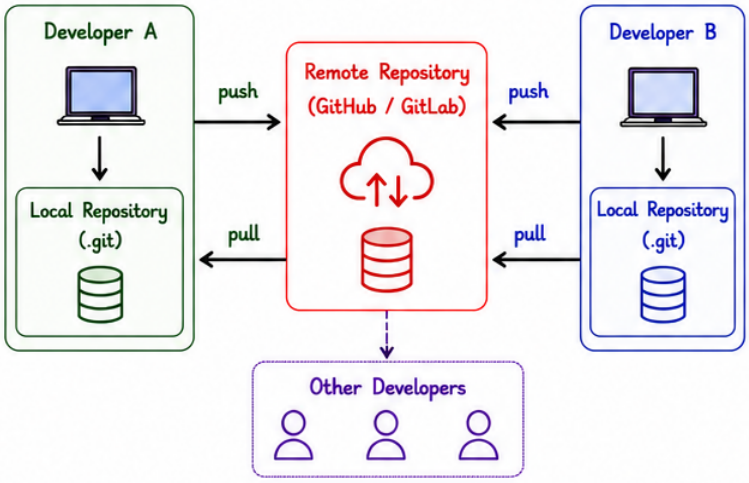
2.1 What is Git Architecture?

Git follows a distributed architecture. Every developer has a full copy of the repository (including history). This allows offline work, fast operations, and secure backups.

Key Concept

- Git is a Distributed Version Control System (DVCS).
- Each developer has a complete local repository.
- Changes are committed locally and pushed to remote.
- Multiple remotes can exist.

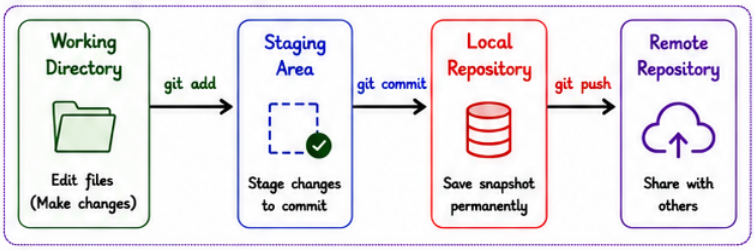
2.2 Architecture Overview



2.3 Main Components

Component	Description
Local Repository	Complete copy of the project on your local machine.
Remote Repository	Central repository hosted on GitHub, GitLab, Bitbucket, etc.
Clone	Creates a copy of remote repo to local machine.
Fetch	Downloads changes from remote but does not merge.
Pull	Fetches and merges changes from remote.
Push	Uploads local commits to remote repository.

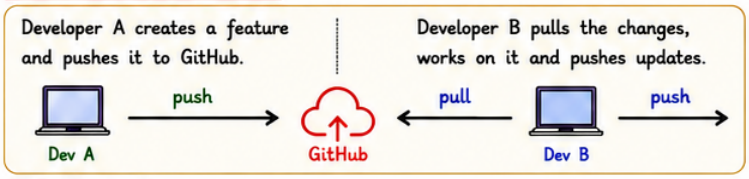
2.4 Data Flow in Git



2.5 Advantages of Git Architecture

- Every developer has full project history.
- Work offline and commit locally.
- Fast operations (no need for constant server).
- No single point of failure.
- Easy branching and experiment safely.
- Better collaboration and code sharing.

2.6 Example Scenario



2.7 Interview Tip

If the remote server goes down, you can still commit, branch, and work locally because Git is Distributed.

2.8 Remember

Git = DVCS | Local Repository = Complete history | Commit Local → Push Remote | Pull = Fetch + Merge

3. INSTALLATION & CONFIGURATION

3.1 Install Git

- Download Git from the official website: https://git-scm.com/downloads
- Install for your OS (Windows / macOS / Linux).
- Verify installation.

Operating System	Installation
Windows	Run the installer (.exe) and follow the setup.
macOS	Use Homebrew: `brew install git`
Linux	Use package manager: `sudo apt install git`

Verify Installation

```
git --version
```

Example Output:

```
git version 2.44.0
```

3.2 Configure Git (Set Your Identity)

Before using Git, configure your username and email.

Command	Description
`git config --global user.name "Your Name"`	Sets your name.
`git config --global user.email "you@email.com"`	Sets your email.
`git config --global core.editor "code --wait"`	Sets default editor (VS Code in this case).

Check Configuration

```
git config --list
```

Displays all configured settings.

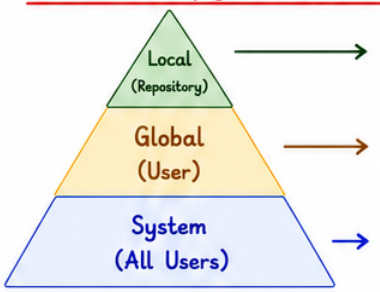
Example Output:

```
user.name=Your Name
user.email=you@email.com
core.editor=code --wait
```

3.3 Helpful Global Configurations

Command	Purpose
`git config --global init.defaultBranch main`	Set default branch name to main.
`git config --global core.autocrlf true`	Handle line ending issues (Windows).
`git config --global color.ui true`	Enable colored output in terminal.
`git config --global pull.rebase false`	Use merge by default when pulling.

3.4 Git Configuration Levels



- Applies to a single repository. Stored in .git/config
- Highest priority

- Applies to current user. Stored in ~/.gitconfig
- Medium priority

- Applies to all users. Stored in /etc/gitconfig
- Lowest priority

3.5 Example Workflow

Install Git (Download & Install) → Verify (`git --version`) → Configure (Set username & email) → Ready to Use (Start using Git!)

3.6 Interview Tip

Always configure Git with your correct name and email. Commit history will reflect this identity.

3.7 Remember

- ✓ Install Git
- ✓ Verify Version
- ✓ Configure Username
- ✓ Configure Email
- ✓ Check Configuration
- ✓ Understand Config Levels
- ✓ You are now ready to use Git!

4. ESSENTIAL GIT COMMANDS

4.1 Initialize Repository

Purpose

Creates a new Git repository in your local folder.

Example

```
$ git init
Initialized empty Git repository in C:/Project/.git/
```

Syntax

```
git init
```

Notes

It creates a hidden .git folder that stores all repository data.

4.2 Check Status

Purpose

Shows the current status of changes in the working directory and staging area.

Example

```
$ git status
On branch main

Changes to be committed:
(use "git restore --staged <file>..." to unstage)

    new file:   index.html
```

Syntax

```
git status
```

Notes

Displays modified, staged, and untracked files.

4.3 Add Files to Staging Area

Command	Syntax	Purpose	Example
Add specific file	`git add <file>`	Adds a specific file to staging area.	`$ git add index.html`
Add all files	`git add .`	Adds all files in the current directory.	`$ git add .`
Add all (including subdirectories)	`git add -A`	Adds all files including deleted/modified ones in the entire project.	`$ git add -A`

4.4 Commit Changes

Purpose

Saves the staged changes to the local repository with a message.

Syntax

```
git commit -m "message"
```

Example

```
$ git commit -m "Initial commit"
[main (root-commit) a1b2c3d] Initial commit
1 file changed, 10 insertions(+)
create mode 100644 index.html
```

Notes

- Every commit should have a meaningful message.
- Commits are stored permanently in history.

4.5 View Commit History

Syntax

```
git log
```

Example Output

```
$ git log --oneline
a1b2c3d (HEAD -> main) Initial commit
b2c3d4e Added index.html
c3d4e5f Added style.css
```

Common Options

`--oneline`	Short output
`--graph`	Show branch graph
`--all`	Show all branches
`--decorate`	Show refs (HEAD, branch)

Notes

Helps to see project history clearly.

4.6 Remove Files

Command	Syntax	Purpose	Example
Remove file (keep locally)	`git rm <file>`	Removes file from staging and working directory.	`$ git rm index.html`
Remove file (keep in disk)	`git rm --cached <file>`	Removes file from staging but keeps in working dir.	`$ git rm --cached index.html`
Remove directory	`git rm -r <dir>`	Removes a directory and all files inside.	`$ git rm -r assets/`

4.7 Rename / Move Files

Syntax

```
git mv <old_name> <new_name>
```

Example

```
$ git mv old.txt new.txt
```

Notes

Git tracks the rename as a move (not as delete + add).

4.8 Remember

git init	git status	git add .	git commit -m "msg"	git log --oneline
Initialize repository	Check current status	Stage all changes	Commit with message	View history

5. BRANCHING & MERGING

5.1 What is a Branch?

A branch is an independent line of development. It allows you to work on new features or fixes without affecting the main code.

Key Points

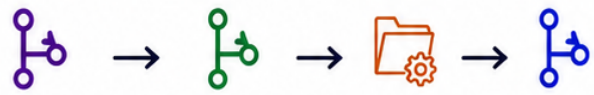
- Default branch name: main (or master)
- You can create multiple branches.
- Branches help in safe development and experiments.

5.2 Branching Commands

Command	Syntax	Purpose	Example
Create a new branch	git branch <branch_name>	Creates a new branch.	$ git branch feature
List branches	git branch	Lists all local branches.	$ git branch
Switch branch	git checkout <branch_name>	Switches to the specified branch.	$ git checkout feature
Create & switch (new way)	git switch -c <branch_name>	Creates a new branch and switches to it.	$ git switch -c feature

5.3 Example: Creating and Switching Branch

1. Create Branch → 2. Switch to Branch → 3. Work on Branch → 4. Back to Main



$ git branch feature

$ git checkout feature

Make changes and commit.

$ git checkout main

5.4 Merge Branch

Purpose

Merging combines the changes from one branch into another branch.

Syntax

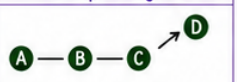
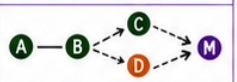
```
git checkout <target_branch>
git merge <source_branch>
```

Example

```
$ git checkout main
$ git merge feature
```

Merges 'feature' branch into 'main' branch.

5.5 Merge Types

Merge Type	Description	Example Diagram
Fast Forward Merge	No new merge commit is created. Just moves the branch pointer forward.	
3-Way Merge	A new merge commit is created when branches have diverged.	
Merge Conflict	Occurs when same lines are changed in both branches.	

5.6 View Branch Graph

Syntax

```
git log --oneline --graph --all
```

Example Output

```
*   7f8a2d4 (HEAD -> main) Merge branch 'feature'
|\
| * 3c2b1e4 (feature) Add login validation
| * 9b8a7c6 Improve UI
* | a1b2c3d Update README
* | 4d5e6f7 Initial commit
```

Notes

- ✓ Always merge into target branch (e.g., main).
- ✓ Resolve conflicts if any, then commit.
- ✓ Delete branch after merge if not needed.

5.7 Delete Branch

Command	Syntax	Purpose	Example
Delete local branch	git branch -d <branch_name>	Deletes the branch (if merged).	$ git branch -d feature
Force delete local branch	git branch -D <branch_name>	Deletes the branch even if not merged.	$ git branch -D feature
Delete remote branch	git push origin --delete <branch_name>	Deletes the branch from remote.	$ git push origin --delete feature

5.8 Interview Tip

Branches help in parallel development. Always pull latest changes before creating a new branch and merge frequently to avoid conflicts.

5.9 Remember

- **Branch** — Create separate line of work
- **Merge** — Combine changes safely
- **View** — Use log --graph to visualize
- **Delete** — Clean unused branches
- **Best Practice** — Keep main stable, review before merge

6. REMOTES & GITHUB BASICS

6.1 What is a Remote?

A remote is a version of your project that is hosted on the internet (e.g., GitHub, GitLab, Bitbucket) so that others can collaborate with you.

Key Points

- Allows code sharing.
- Enables team collaboration.
- Acts as a backup.
- Accessible from anywhere.

6.2 Remote Commands

Command	Syntax	Purpose	Example
Add Remote	`git remote add <name> <url>`	Adds a new remote repository.	`$ git remote add origin <url>`
List Remotes	`git remote -v`	Shows all remotes with URLs.	`$ git remote -v`
Remove Remote	`git remote remove <name>`	Removes a remote.	`$ git remote remove origin`
Rename Remote	`git remote rename <old> <new>`	Renames a remote.	`$ git remote rename origin upstream`
Change Remote URL	`git remote set-url <name> <url>`	Updates URL of a remote.	`$ git remote set-url origin <new-url>`

6.3 Example: Add Remote (GitHub)

Steps

1. Create repo on GitHub
2. Copy HTTPS URL
3. Add Remote in Git
4. Push Code to GitHub

Example

```
$ git remote add origin https://github.com/username/repo.git
$ git push -u origin main
```

Pushes code to GitHub for the first time.

6.4 Fetch vs Pull vs Push

Command	What it Does	When to Use
Fetch	Downloads changes from remote but does not merge.	When you want to see what others changed.
Pull	Fetches changes and merges them into your current branch.	When you want to update your local code.
Push	Sends your local commits to the remote repository.	When you want to upload your changes to GitHub.

6.5 Push & Pull Commands

Command	Syntax	Purpose	Example
Push	`git push <remote> <branch>`	Uploads local commits to remote branch.	`$ git push origin main`
Pull	`git pull <remote> <branch>`	Fetches and merges remote changes to local branch.	`$ git pull origin main`
Push (Set upstream)	`git push -u <remote> <branch>`	Pushes and sets upstream tracking.	`$ git push -u origin main`

6.6 .gitignore File

Purpose

.gitignore

Tells Git which files or folders to ignore from being tracked.

Example .gitignore

```
# Ignore Python
__pycache__/
*.pyc
.env
# Ignore Logs
logs/
*.log
# Ignore IDE files
.vscode/
.idea/
*.suo
```

Notes

- ✓ Keeps repository clean.
- ✓ Prevents unnecessary files from being pushed.
- ✓ Add .gitignore before committing files.

6.7 Clone a Repository

Purpose

Creates a copy of an existing remote repository on your local machine.

Syntax

```
git clone <url>
```

Example

```
$ git clone https://github.com/username/repo.git
```

6.8 Common Remote Workflow

1. Clone / Add Remote
2. Make Changes Locally
3. Commit Changes
4. Pull Latest Changes
5. Push Your Changes
6. Collaborate with Team

6.9 Interview Tip

Always keep your local repository updated with the remote using "git pull" before starting work. This helps in reducing merge conflicts.

6.10 Remember

☆ Always pull before pushing to avoid conflicts.

- Use Remotes for Collaboration
- Fetch → See
Pull → Update
Push → Share
- Use .gitignore to keep repo clean
- Communicate with Team and Sync Regularly
- Backup your code on Remote (GitHub)

7. STASH, TAGS & ADVANCED TOPICS

7.1 What is Stash?

Stash temporarily saves your uncommitted changes so you can work on something else without committing.

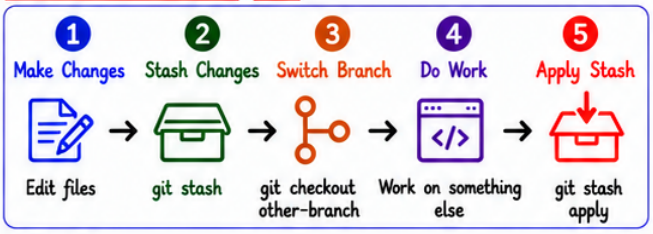
Use Cases

- Work in progress
- Quickly switch branches
- Keep workspace clean

7.2 Stash Commands

Command	Syntax	Purpose	Example
Stash changes	git stash	Stashes the changes in working directory.	$ git stash
Stash with message	git stash push -m "msg"	Stash with a message.	$ git stash push -m "WIP"
List stashes	git stash list	Shows all stashed changes.	$ git stash list
Apply stash	git stash apply	Applies the latest stash without removing.	$ git stash apply
Apply specific stash	git stash apply stash@{n}	Applies specific stash.	$ git stash apply stash@{1}
Drop stash	git stash drop	Deletes the latest stash.	$ git stash drop
Clear all stashes	git stash clear	Removes all stashes.	$ git stash clear

7.3 Stash Workflow



Example

```
$ git remote add origin https://github.com/username/repo.git
$ git push -u origin main
```

Pushes code to GitHub for the first time.

7.4 Fetch vs Pull vs Push

Command	What it Does	When to Use
Fetch	Downloads changes from remote but does not merge.	When you want to see what others changed.
Pull	Fetches changes and merges them into your current branch.	When you want to update your local code.
Push	Sends your local commits to the remote repository.	When you want to upload your changes to GitHub.

7.5 Tag Commands

Command	Syntax	Purpose	Example
Create lightweight tag	git tag <tagname>	Creates a simple tag.	$ git tag v1.0
Create annotated tag	git tag -a <tagname> -m "msg"	Creates tag with message & metadata.	$ git tag -a v1.0 -m "Version 1.0"
List tags	git tag	Lists all tags.	$ git tag
Show tag details	git show <tagname>	Shows tag information.	$ git show v1.0
Push tags	git push origin <tagname>	Push a specific tag.	$ git push origin v1.0
Push all tags	git push --tags	Push all tags.	$ git push --tags

7.6 .gitignore File

Purpose

.gitignore

Tells Git which files or folders to ignore from being tracked.

Example .gitignore

```
# Ignore Python
__pycache__/
*.pyc
.env
# Ignore Logs
logs/
*.log
# Ignore IDE files
.vscode/
.idea/
*.suo
```

Notes

- ✓ Keeps repository clean.
- ✓ Prevents unnecessary files from being pushed.
- ✓ Add .gitignore before committing files.
- ✓ Ignore files are still present locally, just not tracked.

7.8 Cherry-Pick

Cherry-pick applies the changes from one commit to another branch.

Syntax

```
$ git cherry-pick <commit-id>
```

Example

```
$ git cherry-pick a1b2c3d
```

7.7 Undo Changes

Command	Syntax	Purpose	Example
Unstage file	git reset <file>	Removes file from staging area.	$ git reset index.html
Unstage all	git reset	Removes all files from staging area.	$ git reset
Discard changes in working dir	git checkout -- <file>	Discards changes in working directory.	$ git checkout -- index.html
Revert commit (new commit)	git revert <commit-id>	Creates a new commit that undoes changes.	$ git revert a1b2c3d

7.9 Rebase vs Merge

Rebase	Merge
• Creates a merge commit • Preserves full history • Good for collaboration	• Replays commits on top of another branch • Linear history • Use before pushing

Tip: Use Merge for shared branches, Rebase for clean, linear history.

7.10 Interview Tip

Understand the purpose of each command and practice regularly.

Clean history + Clear commits = Professional Git user!

7.11 Remember

Stash to save work temporarily

Use tags for releases

Keep .gitignore updated

Branch & merge carefully

Clean commits always help

Practice commands consistently

7.12 Remember (Key Takeaways)

Stash to save work temporarily

Use tags for releases

Keep .gitignore updated

Branch & merge carefully

Clean commits always help

Practice commands consistently

8. COLLABORATION & TEAMWORK

8.1 Why Collaboration?

Git makes teamwork easy. Multiple developers can work on the same project without overwriting each other's changes.

Benefits

- Work in parallel
- Share changes easily
- Review code
- Track contributions
- Maintain project quality

8.2 Remote Collaboration Workflow

1. Clone Repo — `$ git clone <url>`
2. Create Branch — `$ git checkout -b feature-x`
3. Make Changes
4. Commit & Push

```
$ git add .
$ git commit -m "msg"
$ git push origin feature-x
```

5. Create Pull Request
6. Review & Merge — Merge into main branch

8.3 Clone a Repository

Syntax

```
git clone <repository-url>
```

Example

```
$ git clone https://github.com/user/repo.git
```

Clones the repository to your local machine.

8.4 Forking

Forking creates a copy of someone else's repository in your GitHub account so you can freely experiment and make changes.

Steps to Contribute to a Fork

1. Fork the repository on GitHub
2. Clone your fork: `git clone <your-fork-url>`
3. Create a branch and make changes
4. Push changes to your fork
5. Open Pull Request (PR) to the original repository

8.5 Pull Request (PR)

A Pull Request is a request to merge your changes into the main project. It allows review and discussion before merging.

8.6 Resolve Merge Conflicts

1. Conflict Occurs
2. Open Conflicted File
3. Edit & Resolve Conflicts
4. Save File
5. Commit the Resolved File

Conflict Markers in File

```
<<<<<<< HEAD
Your changes
======
Their changes
>>>>>>> feature-x
```

- HEAD → your current branch
- feature-x → other branch
- Edit the file, keep correct changes and remove the conflict markers.

Best Practices

- Pull latest changes before starting
- Work on feature branches
- Write meaningful commit messages
- Create small & focused PRs
- Review code before merging

8.7 Tracking Contributors

Command	Syntax	Purpose	Example
Show Contributors	`git shortlog -sn`	Shows list of contributors (sorted by commits)	`$ git shortlog -sn`
Detailed Stats	`git log --stat`	Shows changes with details	`$ git log --stat`
Author Summary	`git log --author=<name>`	Shows commits by specific author	`$ git log --author=John`

8.8 Code Review & Feedback

Why Code Review?

- Find bugs early
- Improve code quality
- Share knowledge
- Maintain consistent style
- Build better software together

Tips for Good Reviews

- Be respectful and constructive
- Focus on the code, not the person
- Suggest improvements
- Test the changes locally
- Approve only when satisfied

8.9 Syncing with Remote Repository

Goal	Command	Description	Example
Fetch Updates	`git fetch origin`	Downloads changes without merging	`$ git fetch origin`
Pull Updates	`git pull origin main`	Fetches and merges changes from remote	`$ git pull origin main`
Push Changes	`git push origin <branch>`	Uploads your changes to remote	`$ git push origin feature-x`

8.10 Interview Tip

Highlight your experience with:

- Branching strategy
- Collaboration using PRs
- Conflict resolution
- Code reviews
- Clean commits

This shows you are a team player and follow best practices!

Remember

- Communicate with your team
- Pull before you push
- Resolve conflicts carefully
- Review and test before merging
- Keep history clean and meaningful